

Ejercicio: “Refactor a Funciones Flecha (=>) en JavaScript”

Enunciado del reto

Formas parte de un equipo que estandariza código a ES6. Debes **refactorizar** funciones tradicionales hacia **funciones flecha (=>)**, aplicando reglas de simplificación y confirmando que el comportamiento se mantiene (misma salida para mismas entradas). El foco es escribir **código limpio, legible y reutilizable**.

Según los contenidos del **Módulo 4: Fundamentos de Programación en JavaScript**, la unidad de funciones exige programar funciones con parámetros y retorno, y comprender alcance de variables y modularidad; aquí lo harás usando la sintaxis moderna =>.

Las diapositivas de apoyo indican **qué es una función flecha**, diferencias frente a la forma tradicional y **reglas de transformación** (quitar function, usar =>, retorno implícito en una sola línea y omitir paréntesis con un único parámetro).

Instrucciones

1) Explora el código base (mini-módulos)

Copia en refactorFlecha.js las funciones **tradicionales** del bloque “Base” (ver Sección C). Lee su firma (parámetros, retorno) y el comentario de propósito.

✓ Identifica al menos 1 entrada válida y 1 borde (p. ej., división por 0, string vacío).

2) Refactoriza a funciones flecha

Aplica las reglas de transformación:

- quita function y usa =>;

- si tiene **una** línea, usa **retorno implícito** sin {} ni return;
- si hay **un** parámetro, puedes omitir ()

✓ Cada función refactorizada compila y no rompe el archivo.

3) Verifica equivalencia de resultados

Para cada función, ejecuta los **tests de ejemplo** que vienen en la sección C (console.log con entradas típicas y de borde).

Micro-check: la salida antes y después del refactor es **idéntica**.

4) Mejora de código limpio

- Renombra parámetros poco claros (a, b) por nombres significativos.
- Extrae condiciones repetidas a funciones utilitarias flecha (p. ej., esNumero).
- Añade comentarios breves (qué hace, no cómo).

✓ El archivo queda organizado por secciones y sin duplicación innecesaria.

5) Documenta tu entrega

En la cabecera del archivo, agrega: título, autor, fecha, breve descripción y lista de funciones modificadas.

✓ Ejecuta el archivo completo; no deben aparecer errores en consola.

Criterios de éxito (observables)

- Refactor correcto a => respetando reglas (retorno implícito cuando aplica).
- Resultados equivalentes antes/después en todos los tests provistos.
- Código legible: nombres descriptivos, comentarios breves, organización por secciones.
- Manejo básico de casos borde (por ejemplo, división por cero).

Errores comunes

- Olvidar return al mantener llaves {} en funciones con más de una línea.
- Usar retorno implícito cuando hay **múltiples** sentencias (debe llevar {} y return).
- Omitir paréntesis en flecha con **más de un** parámetro (deben incluirse).